Building a Reactive Database Engine from First Principles

Architectural Research Guide

This document maps the key architectural topics you need to master, which systems to study for each, and the specific resources to dig into. The goal: combine the best ideas from FoundationDB, TigerBeetle, CockroachDB, Convex, Electric SQL, and others into a single-binary reactive database engine written in Rust.

---

1. Embedded Storage Engine (The Foundation Layer)

This is the bottom of the stack — where bytes hit disk. Every decision here cascades upward.

The Core Decision: LSM Tree vs B-Tree vs Copy-on-Write B-Tree

LSM Trees (Log-Structured Merge Trees) are optimized for write-heavy workloads. Writes go to an in-memory buffer (memtable), flush to sorted files on disk (SSTables), and get periodically compacted. Used by RocksDB, Pebble, TigerBeetle, and Cassandra. The tradeoff: fast writes, but reads may need to check multiple levels, and compaction creates background I/O spikes.

B-Trees are the traditional choice (SQLite, Postgres, InnoDB). Reads are predictable (one tree traversal), but writes are more expensive due to random I/O and in-place updates. Well-understood, battle-tested.

Copy-on-Write B-Trees (used by LMDB and redb) never modify existing pages — instead, they write new pages and update pointers. This gives you free snapshots and MVCC without a WAL, at the cost of write amplification from copying interior nodes.

Study These Systems

System	Language	Design	Why Study It
SQLite	C	B-tree, WAL, single-file	The gold standard for embedded databases. Study its WAL mode, page cache, and how it achieves ACID in a single file. Convex's open-source backend already supports SQLite as a persistence layer.
redb	Rust (pure)	Copy-on-write B-tree	A pure-Rust embedded KV store inspired by LMDB. No C dependencies. ACID transactions, MVCC via copy-on-write. This is your most natural fit for a Rust single-binary.

System	Language	Design	Why Study It
Pebble	Go	LSM tree	CockroachDB's storage engine. Study its compaction strategies, bloom filters, and how it optimizes for CockroachDB's specific access patterns. Written in Go, but the design translates.
RocksDB	C++	LSM tree	The industry-standard LSM engine. Study its column families, compaction styles (leveled, universal, FIFO), and rate limiting. Facebook's production workhorse.
TigerBeetle's LSM-Forest	Zig	Custom LSM with deterministic compaction	A novel LSM design where compaction is spread evenly across operations to bound worst-case latencies. Replicas converge on byte-identical disk layouts.
sled	Rust	Lock-free B+ tree	Rust-native, lock-free, uses io_uring. Ambitious design but had stability issues. Study the ideas, be cautious about production readiness.

Key Reading

- SQLite architecture docs: https://www.sqlite.org/arch.html
- redb design doc: https://github.com/cberner/redb (see design doc in repo)
- Pebble introduction: https://www.cockroachlabs.com/blog/pebble-rocksdb-kv-store/
- RocksDB wiki on compaction: https://github.com/facebook/rocksdb/wiki/Compaction
- TigerBeetle LSM-Forest: https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/ARCHITECTURE.md
- "The Log-Structured Merge-Tree" (O'Neil et al., 1996) — the original paper

Recommendation for Your Project

Start with **redb** for the initial implementation. It's pure Rust, zero dependencies, ACID-compliant, and gives you copy-on-write snapshots for free (essential for MVCC and reactive queries). If write throughput becomes the bottleneck, consider building a custom LSM layer later, informed by TigerBeetle's deterministic compaction design. Avoid RocksDB (C++ FFI complexity) and sled (uncertain future) for a v1.

---

2. Transaction Manager & Concurrency Control

This layer ensures ACID guarantees. The choice here determines your consistency model and directly affects reactive query feasibility.

The Core Decision: OCC vs MVCC vs Deterministic Transactions

Optimistic Concurrency Control (OCC) — used by Convex and FoundationDB. Transactions execute without locks, then validate at commit time. If a conflict is detected, the transaction is retried. Great for read-heavy workloads and reactive systems because reads never block.

Multi-Version Concurrency Control (MVCC) — used by CockroachDB, Postgres, and most modern databases. Each write creates a new version of the data, and reads see a consistent snapshot. Enables snapshot isolation and serializable isolation. More complex to implement but well-understood.

Deterministic Transactions — used by Calvin, FaunaDB, and TigerBeetle. Transaction order is determined before execution, eliminating the need for concurrency control during execution. Extremely high throughput but requires knowing the read/write set upfront.

Study These Systems

System	Approach	Why Study It
Convex	OCC with serializable isolation	Study how they combine OCC with reactive query invalidation. When a transaction commits, Convex knows exactly which queries to re-evaluate because OCC tracks read sets. This is the key insight for reactive systems.
FoundationDB	OCC + MVCC hybrid, strict serializability	Study their 5-second transaction time limit (forces simple transactions), their conflict detection via read ranges, and how they decouple the write path (transaction system) from the read path (storage servers).
CockroachDB	MVCC with HLC timestamps	Study their hybrid logical clocks for ordering across distributed nodes, their timestamp cache for detecting conflicts, and how they achieve serializable isolation without a single sequencer.
TigerBeetle	Deterministic, single-writer	Study their radical simplification: strict single-threading eliminates all concurrency concerns within a node. The consensus protocol handles ordering across nodes.
Calvin (paper)	Deterministic transactions	The foundational paper on deterministic databases. Pre-determines transaction order in a log, then executes deterministically. Eliminates distributed commit protocols.

Key Reading

- Convex transaction model: https://docs.convex.dev/understanding/
- FoundationDB paper (SIGMOD 2021): https://www.foundationdb.org/files/fdb-paper.pdf
- CockroachDB design doc (transactions section): https://github.com/cockroachdb/cockroach/blob/master/docs/design.md
- TigerBeetle design doc: https://github.com/tigerbeetle/tigerbeetle-history-archive/blob/main/docs/DESIGN.md
- "Calvin: Fast Distributed Transactions for Partitioned Database Systems" (Thomson et al., 2012)
- "A Critique of ANSI SQL Isolation Levels" (Berenson et al., 1995) — understand what you're implementing
- "Serializable Snapshot Isolation" (Cahill et al., 2009)

Recommendation for Your Project

Use **OCC with serializable isolation**, following Convex's model. OCC is the natural fit for a reactive system because: (a) read sets tracked during OCC validation are exactly the dependency information you need to invalidate reactive queries, (b) reads never block, keeping subscription latency low, and (c) it's simpler to implement correctly than MVCC for a single-node system. Layer MVCC on top later if you need time-travel queries or snapshot reads.

---

3. Reactive Query Engine (The Differentiator)

This is what makes the system special — queries that automatically push updates to clients when underlying data changes. This is the hardest part to get right.

The Core Decision: Full Re-evaluation vs Incremental View Maintenance

Full re-evaluation — when data changes, re-run the entire query. Simple to implement, but O(N) per change where N is the result set size. This is what Convex does today.

Incremental View Maintenance (IVM) — when data changes, compute only the delta to the query result. Can be O(1) per change for simple queries. Much harder to implement but dramatically more efficient at scale. Materialize/Differential Dataflow pioneered this approach.

Study These Systems

System	Approach	Why Study It
Convex	Full re-evaluation with dependency tracking	Study how they track which tables/documents a query function reads, and use OCC conflict detection to know when to re-run queries. Simple, correct, scales to moderate subscription counts.
Materialize / **Differential Dataflow**	Incremental view maintenance via dataflow	Study Frank McSherry's Differential Dataflow — the most rigorous approach to incremental computation. Queries are compiled into dataflow graphs where changes propagate as deltas. Handles joins, aggregations, and window functions incrementally.
Noria (MIT research)	Partial stateful dataflow	A streaming dataflow system that maintains partial state — only materializing parts of views that are actually read. Handles "upquery" (backward queries to fill missing state). Study the tradeoffs of partial vs full materialization.
RisingWave	Streaming SQL with IVM	A cloud-native streaming database that maintains materialized views incrementally. Study their approach to watermarks, consistency, and how they handle late-arriving data.
Firebase / **Firestore**	Document-level subscriptions	Study the simplest version of the problem: subscribe to individual documents or simple queries, get notified on change. Understand why this model hits limits for complex queries.
SpacetimeDB	Co-located compute and storage, IVM	Study their approach to incremental query evaluation and how co-locating compute with storage eliminates network hops for subscription updates.

Key Reading

- "Naiad: A Timely Dataflow System" (Murray et al., 2013) — the foundation for Differential Dataflow
- Differential Dataflow: https://github.com/TimelyDataflow/differential-dataflow
- Frank McSherry's blog: https://github.com/frankmcsherry/blog — deep dives on IVM
- Noria paper: "Noria: Dynamic, Partially-Stateful Data-Flow for High-Performance Web Applications" (Gjengset et al., 2018)

- Convex reactive internals: https://stack.convex.dev/ (search for "reactive" and "subscriptions")
- "Riffle: Building data-centric apps with a reactive relational database" — covers reactivity and performance
- Materialize architecture: https://materialize.com/docs/overview/

Recommendation for Your Project

Start with **Convex-style full re-evaluation with dependency tracking** for the v1. It's dramatically simpler to implement correctly. Track read sets per query (which tables, which index ranges, which documents were accessed). When a write commits, compare its write set against all active queries' read sets — any intersection means the query needs re-evaluation. Push the new result to the client.

For v2, move toward **incremental view maintenance** for hot queries. Compile frequently-subscribed queries into mini dataflow graphs that process change deltas. This is where studying Differential Dataflow and Noria pays off. The key insight from Noria is that you don't need to incrementally maintain everything — only the views that are actually subscribed to.

---

4. Sync Protocol & Client State Management

How data moves between server and clients, and how clients maintain a consistent local view.

Study These Systems

System	Approach	Why Study It
Convex	Server-authoritative, WebSocket subscriptions	Study their model where the server is the source of truth and pushes consistent snapshots to clients. No client-side conflict resolution needed. Simple and correct.
Electric SQL	Bidirectional sync, CRDTs, Postgres + SQLite	Study their "shapes" API for partial replication (sync only the data you need). Their use of Postgres logical replication (WAL) to detect changes. Their CRDT-based conflict resolution for offline writes. Built by the inventors of CRDTs.
Replicache / Zero	Client-side optimistic mutations, server reconciliation	Study their "mutator" model where the client speculatively applies changes locally, sends them to the server, and reconciles when the server confirms or rejects. Great UX (instant local updates) with strong eventual consistency.

System	Approach	Why Study It
Linear	Incremental sync engine	Study how Linear syncs workspace data to the client for instant UI. Their sync engine handles partial sync, incremental updates, and offline support.
PowerSync	Server-authoritative writes, client-side reads from SQLite	Study their separation of read and write paths: reads come from local SQLite (fast), writes go through your backend (flexible conflict resolution).
Litestream	SQLite replication to S3	Study their approach to WAL-based replication of SQLite databases to object storage. Relevant if you use SQLite as your embedded storage.

Key Reading

- Electric SQL architecture: https://electric-sql.com/blog/2023/02/09/developing-local-first-software
- Replicache design: https://doc.replicache.dev/
- "Local-first software: You own your data, in spite of the cloud" (Ink & Switch, 2019) — the foundational essay
- CRDTs overview: "Conflict-free Replicated Data Types" (Shapiro et al., 2011)
- "A Comprehensive Study of CRDTs" (Shapiro et al., 2011)
- Zero (from Replicache team): https://zero.rocicorp.dev/

Recommendation for Your Project

For v1, use **Convex's server-authoritative model** with WebSocket subscriptions. The server is the source of truth; clients receive consistent snapshots. This avoids the enormous complexity of CRDTs and conflict resolution. Support optimistic updates on the client side (speculatively apply the mutation, then confirm/reject when the server responds).

For v2, add **offline support with a local embedded database** (SQLite or your own storage engine compiled to WASM). Queue writes when offline, replay them when reconnected. Use a simple last-writer-wins or application-defined conflict resolution strategy rather than full CRDTs — most applications don't need arbitrary concurrent edits.

---

5. Consensus & Replication (For Multi-Node)

Not needed for a single-node v1, but essential to design for from the start so you don't paint yourself into a corner.

Study These Systems

System	Protocol	Why Study It
FoundationDB	Active Disk Paxos + custom coordination	Study their "unbundled" architecture: transaction management, logging, and storage are separate roles that scale independently. Study their recovery protocol — unusually fast because of deterministic transaction ordering.
CockroachDB	Raft per range	Study how they use one Raft group per range (data shard), with leaseholders for read locality. Understand the overhead of per-range Raft and when it matters.
TigerBeetle	Viewstamped Replication (VSR)	Study VSR as an alternative to Raft/Paxos. TigerBeetle chose it because leader election is deterministic. Study their Flexible Quorums optimization (reducing the cost of synchronous replication). Study their protocol-aware recovery for consensus-based storage.
etcd	Raft	The canonical Raft implementation. Study the Raft paper first, then etcd's implementation.
Raft paper	—	"In Search of an Understandable Consensus Algorithm" (Ongaro & Ousterhout, 2014). The clearest explanation of consensus. Read this before studying any specific implementation.

Key Reading

- Raft paper: https://raft.github.io/raft.pdf
- FoundationDB paper (recovery section): https://www.foundationdb.org/files/fdb-paper.pdf
- TigerBeetle VSR: https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/ARCHITECTURE.md
- "Viewstamped Replication Revisited" (Liskov & Cowling, 2012)
- "Flexible Paxos: Quorum Intersection Revisited" (Howard et al., 2016) — the paper behind TigerBeetle's Flexible Quorums
- "Protocol-Aware Recovery for Consensus-Based Storage" (CTRL protocol)

Recommendation for Your Project

Design the storage and transaction layers to be **deterministic from day one** — this makes adding replication later dramatically simpler. If your state machine is deterministic (same inputs → same

outputs → same disk layout), then replication reduces to agreeing on the input log, which is exactly what Raft/VSR solve.

When you're ready for multi-node, start with **Raft** (best ecosystem and tooling in Rust — see `openraft` or `raft-rs`), then consider VSR if you need TigerBeetle-style physical determinism and self-healing storage.

---

6. I/O Architecture & Mechanical Sympathy

How you interact with disk and network at the syscall level. This determines your performance ceiling.

Study These Systems

System	Approach	Why Study It
TigerBeetle	io_uring for everything, zero-copy, static allocation, single-threaded event loop	The most extreme example of mechanical sympathy in a modern database. Study their unified io_uring abstraction for both disk and network. Study their static memory allocation (no malloc at runtime). Study their cache-line-aligned data structures.
FoundationDB	Flow (custom actor framework), simulated I/O	Study how they abstract I/O behind interfaces that can be swapped for simulation. The real I/O layer uses Boost.ASIO; the simulated layer runs everything in a single thread.
Tokio (Rust)	Async runtime with io_uring support	The Rust ecosystem's standard async runtime. Study its work-stealing scheduler and `tokio-uring` for io_uring integration. For a database, you may want a more controlled event loop than Tokio provides.
Monoio / **Glommio** (Rust)	Thread-per-core, io_uring-native	Rust async runtimes designed specifically for io_uring with a thread-per-core model (no work stealing). Closer to TigerBeetle's philosophy. Study these as alternatives to Tokio for database workloads.

Key Reading

- TigerBeetle io_uring blog: https://tigerbeetle.com/blog/2022-11-23-a-friendly-abstraction-over-iouring-and-kqueue/

- "A New Era for Database Design with TigerBeetle" (InfoQ talk): https://www.infoq.com/presentations/tigerbeetle/
- Martin Thompson's "Mechanical Sympathy" talks (YouTube) — the intellectual foundation for TigerBeetle's design
- io_uring intro: "Efficient IO with io_uring" (Axboe, 2019)
- TigerStyle coding guidelines: https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/TIGER_STYLE.md

Recommendation for Your Project

Use **io_uring for disk I/O** (with kqueue fallback on macOS) and **a controlled event loop** rather than a general-purpose async runtime. For networking, Tokio is fine for the WebSocket/HTTP layer — it doesn't need the same precision as disk I/O. Adopt TigerBeetle's principle of explicit, static memory allocation for the hot path (storage engine, query evaluation) while using standard Rust allocation for the cold path (connection setup, schema compilation).

---

7. Deterministic Simulation Testing

How you achieve confidence that your system is correct under all failure conditions. This is arguably the most important architectural decision.

Study These Systems

System	Approach	Why Study It
FoundationDB	Custom simulation framework (Flow + Simulation), BUGGIFY macro, trillion CPU-hours of testing	The originator of deterministic simulation for databases. Study how they abstract all I/O behind swappable interfaces, how BUGGIFY injects rare code paths, and how their test oracles verify correctness. They built the simulator before the database.
TigerBeetle	The VOPR (Viewstamped Operation Replicator), runs entire cluster in single thread	Study how they simulate network faults, storage corruption, and process failures. Their hash-chained state checker verifies all state transitions are valid. They can inject storage faults at rates up to the theoretical limit.

System	Approach	Why Study It
Antithesis (company)	Deterministic hypervisor for any software	Founded by the FoundationDB simulation team. Study their approach of running entire Docker containers under a deterministic hypervisor. WarpStream, MongoDB, and others use it. Relevant as an external testing tool even if you build your own simulator.
WarpStream	Uses Antithesis to test their entire SaaS	Study how they simulate not just the database but the entire deployment, including signup flows and Kafka workloads.

Key Reading

- FoundationDB simulation docs: https://apple.github.io/foundationdb/testing.html
- "Diving into FoundationDB's Simulation Framework" (Pierre Zemb): https://pierrezemb.fr/posts/diving-into-foundationdb-simulation/
- FoundationDB CACM article (simulation section): https://cacm.acm.org/research-highlights/foundationdb-a-distributed-key-value-store/
- TigerBeetle VOPR: https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/ARCHITECTURE.md
- Antithesis: https://antithesis.com/docs/resources/deterministic_simulation_testing/
- WarpStream DST blog: https://www.warpstream.com/blog/deterministic-simulation-testing-for-our-entire-saas
- Will Wilson's "Testing Distributed Systems w/ Deterministic Simulation" (Strange Loop talk, YouTube)
- `turmoil` (Rust): https://github.com/tokio-rs/turmoil — a Rust framework for deterministic simulation testing of Tokio-based systems

Recommendation for Your Project

Build the simulation framework before building the database. This is the single most important lesson from both FoundationDB and TigerBeetle. Abstract all I/O (disk, network, time) behind traits from day one. Make every source of non-determinism (random number generation, timestamps, thread scheduling) injectable. Use a seeded PRNG so any failure is perfectly reproducible.

In Rust, this means defining traits like `trait Storage`, `trait Network`, `trait Clock` and having both real implementations and simulated implementations. Use `turmoil` as inspiration, or build a custom simulator if `turmoil` doesn't give you enough control over storage faults.

8. Schema-Driven API & User-Defined Logic

How developers define their data model and business logic, and how the system derives an API from it.

Study These Systems

System	Approach	Why Study It
Convex	TypeScript functions (queries, mutations, actions) running in V8 isolates inside the database	Study how they execute user code deterministically, how they sandbox it, and how they achieve the "functions as transactions" model.
Hasura	Auto-generated GraphQL API from Postgres schema	Study their approach to deriving a complete CRUD API with auth, filtering, pagination, and subscriptions directly from the database schema. No user code needed for basic operations.
PostgREST	Auto-generated REST API from Postgres schema	Simpler than Hasura but same philosophy. Study how they map HTTP verbs to SQL operations and how they use Postgres row-level security for auth.
Supabase	Postgres + PostgREST + Realtime (via logical replication)	Study how they combine schema-driven API generation with real-time subscriptions via Postgres's WAL.
Cloudflare Workers / WASM	User code runs in V8 isolates or WASM sandboxes	Study how they achieve millisecond cold starts, memory isolation, and deterministic execution. Relevant for running user-defined functions inside your database process.

Key Reading

- Convex function model: https://docs.convex.dev/functions
- Hasura architecture: https://hasura.io/docs/latest/getting-started/overview/
- PostgREST: https://postgrest.org/
- Wasmtime (Rust WASM runtime): https://wasmtime.dev/
- V8 embedding: https://v8.dev/docs/embed

- "Convex: The Software-Defined Database": https://stack.convex.dev/the-software-defined-database

Recommendation for Your Project

Take a **two-tier approach**: (1) Auto-generate a typed API from the schema for basic CRUD and subscriptions (Hasura/PostgREST style), so developers get an instant API without writing any server code. (2) Allow custom logic via **WASM plugins** for complex business rules, computed fields, and triggers. WASM gives you language-agnostic sandboxing (TypeScript, Rust, Go, Python can all compile to WASM) without embedding a full V8 runtime. Use `wasmtime` in Rust — it's production-grade and gives you deterministic execution.

---

9. Auth & Access Control

How you secure data at the database level rather than the application level.

Study These Systems

System	Approach	Why Study It
Postgres	Row-Level Security (RLS) policies	The gold standard for database-level access control. Study how RLS policies are evaluated as part of the query planner, making them impossible to bypass.
Supabase	Postgres RLS + JWT-based auth	Study how they combine RLS with JWT tokens to create per-user data views without application-level middleware.
Convex	Auth checked in user functions	Study their model where auth is a first-class argument to every function. Simple but means auth is enforced in application code, not the database.
Firebase Security Rules	Declarative rules evaluated at the database layer	Study their expression language for access control. Rules reference the auth token, the requested data, and existing data. Evaluated atomically with the read/write.
Zanzibar (Google)	Relationship-based access control (ReBAC)	The system behind Google Drive, YouTube, and Cloud permissions. Study how they model permissions as relationships in a graph and evaluate access with graph traversal.

Key Reading

- Postgres RLS: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

- Firebase Security Rules: https://firebase.google.com/docs/rules
- "Zanzibar: Google's Consistent, Global Authorization System" (Pang et al., 2019)
- Oso (authorization library): https://www.osohq.com/
- Cerbos: https://www.cerbos.dev/

Recommendation for Your Project

Implement **schema-level access control** that is evaluated inside the query engine, not in a middleware layer. Define permissions declaratively in the schema (similar to Firebase Security Rules but with SQL-like expressions). The key requirement: a query can never return data the user isn't authorized to see, regardless of how the query is constructed. This means access control must be part of the query planner, not a post-filter.

---

10. Architecture Summary: The Best Combinations

Here's how the best ideas from each system combine into a coherent architecture:

Layer	Best-in-Class Source	Approach
Storage Engine	redb + TigerBeetle ideas	Pure Rust, copy-on-write B-tree (redb) for v1. Add deterministic compaction (TigerBeetle) if moving to LSM later.
Transaction Manager	Convex + FoundationDB	OCC with serializable isolation. Track read/write sets for both conflict detection AND reactive query invalidation.
Reactive Queries	Convex (v1) → Materialize (v2)	Full re-evaluation with dependency tracking first. Incremental view maintenance for hot paths later.
Sync Protocol	Convex model + Replicache ideas	Server-authoritative with WebSocket push. Client-side optimistic mutations with server confirmation.
Consensus	Raft (via openraft) or VSR (TigerBeetle)	Not needed for single-node v1, but design for determinism from day one so adding replication is straightforward.
I/O Layer	TigerBeetle	io_uring for disk, explicit memory management on hot paths, single-threaded event loop for the storage engine.
Testing	FoundationDB + TigerBeetle	Deterministic simulation with swappable I/O interfaces. Build the simulator first.
Schema/API	Hasura + Convex	Auto-generated typed API from schema, with WASM plugins for custom logic.
Auth	Postgres RLS + Firebase Rules	Declarative access control evaluated inside the query engine.
Language	Rust	Single static binary, no runtime, no GC, fearless concurrency, C ABI for embedding.

Priority Reading Order

If you're going to read things in order, this sequence builds understanding from the ground up:

1. **Raft paper** — understand consensus before anything else
2. **FoundationDB SIGMOD paper** — the best overview of how a serious distributed database is architected

3. **TigerBeetle ARCHITECTURE.md** — the most modern take on first-principles database design
4. **Convex docs** (understanding section) — how reactive queries + OCC + TypeScript functions fit together
5. **Differential Dataflow** (Frank McSherry's blog) — the theory behind incremental view maintenance
6. **"Local-first software"** (Ink & Switch) — the philosophical foundation for client-side sync
7. **Pierre Zemb's FoundationDB simulation deep dive** — how to actually build a deterministic simulator
8. **TigerBeetle InfoQ talk** — Joran Dirk Greef explaining the "new era" of database design
9. **CockroachDB design.md** — how to layer SQL on top of a distributed KV store
10. **Electric SQL architecture posts** — CRDTs, sync engines, and the realities of local-first

---

Open Source Codebases to Read

Repo	Language	Lines	License	Focus
`get-convex/convex-backend`	Rust + TS	~200k	FSL → Apache 2.0	Reactive queries, OCC, function execution
`tigerbeetle/tigerbeetle`	Zig	~100k	Apache 2.0	Storage engine, consensus, simulation
`apple/foundationdb`	C++ (Flow)	~500k	Apache 2.0	Simulation, distributed transactions, recovery
`cockroachdb/cockroach`	Go	~1M+	BSL → Apache 2.0	Distributed SQL, Raft, query planning
`cockroachdb/pebble`	Go	~90k	BSD	LSM tree implementation
`cberner/redb`	Rust	~15k	MIT / Apache 2.0	Embedded KV store, copy-on-write B-tree
`MaterializeInc/materialize`	Rust	~500k	BSL	Incremental view maintenance, Differential Dataflow
`electric-sql/electric`	Elixir + TS	~50k	Apache 2.0	Sync engine, shapes API, CRDT integration

Repo	Language	Lines	License	Focus
`nickel-org/turmoil`	Rust	~5k	MIT	Deterministic simulation for Tokio
`tokio-rs/tokio`	Rust	~100k	MIT	Async runtime, io_uring support

Glossary of Key Concepts

OCC — Optimistic Concurrency Control. Execute without locks, validate at commit. **MVCC** — Multi-Version Concurrency Control. Multiple versions of data coexist for snapshot reads. **IVM** — Incremental View Maintenance. Updating a materialized view by processing only the changes. **LSM Tree** — Log-Structured Merge Tree. Write-optimized storage with background compaction. **WAL** — Write-Ahead Log. Durability mechanism: log the change before applying it. **CRDT** — Conflict-free Replicated Data Type. Data structures that merge without coordination. **VSR** — Viewstamped Replication. A consensus protocol alternative to Raft/Paxos. **HLC** — Hybrid Logical Clock. Combines physical and logical clocks for distributed ordering. **io_uring** — Linux kernel interface for asynchronous I/O without syscall overhead. **DST** — Deterministic Simulation Testing. Testing real code under simulated faults with perfect reproducibility. **BUGGIFY** — FoundationDB's macro for injecting rare code paths during simulation. **VOPR** — TigerBeetle's deterministic cluster simulator.